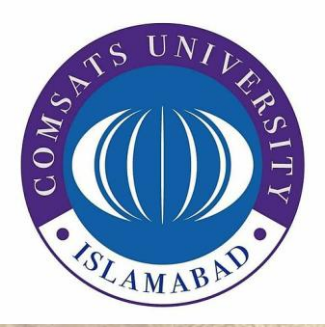




SOFTWARE MAINTENANCE MATURITY MODEL FOR OPERATION PHASE

Ms.Areej Bashir

Lecturer, CUI ATD.

CONTENT



- Software Maintenance Service
- Software Maintenance Service Foundations
- Responsibilities of Maintenance Staff
- Managerial processes of software operation quality assurance

SOFTWARE MAINTENANCE SERVICES OBJECTIVES



- Ensure, with an accepted level of confidence, that the software maintenance activities conform to functional technical requirements.
- Ensure, with an accepted level of confidence, that the software maintenance activities conform to scheduling and budgetary requirements.
- Initiate and manage activities to improve the effectiveness and increase the efficiency of software maintenance activities. These activities improve the prospects of
- achieving functional and managerial requirements, while reducing costs.

SOFTWARE MAINTENANCE SERVICE FOUNDATIONS



1. **Software package quality.** It goes without saying that the quality of the new software package is perhaps the single most important foundation underlying the quality of user support and maintenance services.
2. **Software release and software version policies.** Other critical foundations are the software releases and software version policies.
3. **Specific QA procedures for operation services.** The third foundation of software operation quality is the specific QA procedures, which the software development organization operates.

FOUNDATION TWO—SOFTWARE RELEASES AND VERSION POLICIES



- The frequency of software version releases—policy:

The typical frequency is once a year, though twice a year and once in two years is also found. The perfective maintenance team may release one or more revisions between successive version releases. Software version releases include major changes and additions to the software system, while revision releases include minor changes and urgent software corrections.

THE LINEAR EVOLUTION MODEL:



- ☐ In this model, only one version of the software is available at any time.
- ☐ When a new version is released, it replaces the old version completely.
- ☐ This model is best for:
 - Organizations with a single set of needs.
 - Software that is used by everyone in the same way (e.g., a popular app with uniform features for all users).
- ☐ The benefit: It is simple to manage and works well when users don't need customized versions.

THE TREE EVOLUTION MODEL:



- ☐ In this model, there are multiple versions of the software being developed and used at the same time.
- ☐ Different versions are made to serve different types of customers or purposes.
- ☐ Example:
 - One version might focus on fixing bugs.
 - Another version might focus on adding new features.
- ☐ The benefit: It is flexible and allows the software to meet the needs of different customers or systems at the same time.
- ☐ This approach is often used for things like firmware (software for hardware devices), where each version might work with a specific device or product line.

COMPARISON



The linear evolution model

According to the linear model, only one unique software system version serves all customers at any given time. Each new version replaces the prior version. This model is the natural choice for software systems developed to serve a single organization. The model is also applied to popular software packages, which tend to be uniform in structure, and where the ability to meet a wide range of maintenance demands in a single version is a great advantage.

The tree evolution model

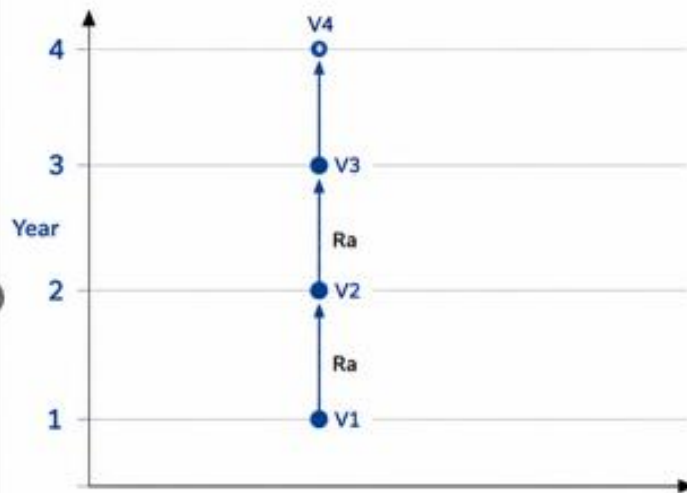
According to this model, several parallel versions of the software are developed to serve the different needs of different customers simultaneously throughout the system's life cycle. Tree models are typically applied in firmware configuration versions, where each branch serves a different product or product line.

COMPARISON



LINEAR EVOLUTION MODEL

Single Version – Sequential Evolution



Description

- One main line of development.
- Each new version is based on the previous version.
- Older version is replaced by the new version.
- Simple to manage.
- Suitable when all users move to the latest version.

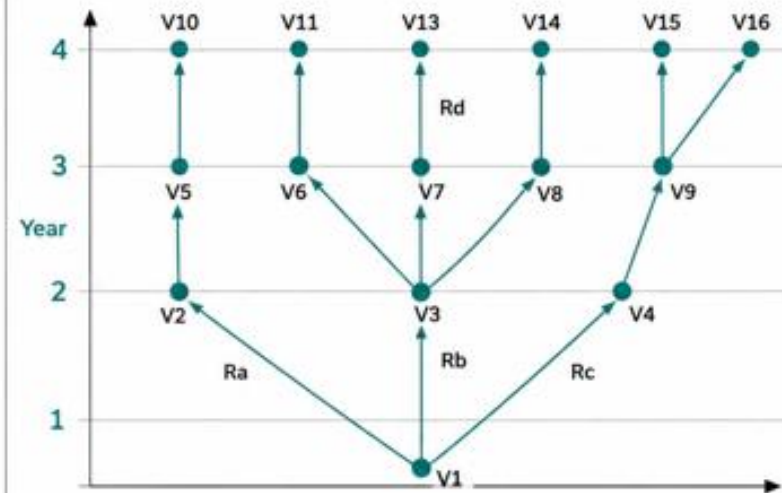
Example



Google Chrome

TREE EVOLUTION MODEL

Multiple Versions – Parallel Evolution



Description

- Multiple branches from a base version.
- Different versions evolve in parallel.
- Supports customer-specific requirements.
- Higher flexibility.
- More complex to manage.

Examples



Android OEM
Firmware



Cisco Firmware

MAINTENANCE TEAM RESPONSIBILITIES:



The daily experience of the maintenance team, therefore, includes overcoming hardships, created by the version-structure of the package, that go beyond those related to the software itself:

- **Faulty corrections** caused by **inadequate identification of the module** structure of the current version used by the specific customer.
- **Faulty corrections** caused by the **incorrect replacement of a faulty** module with a module of a different version that later proved to be unsuitable OR inappropriate to be integrated into the customer's package version.
- **Efforts invested to convince customers** to update their software package by adding newly developed modules or replacing current module versions with a new version. Some customers insisted on keeping their existing version of the package, adding to the number of versions to be maintained.

MANAGERIAL PROCESSES OF SOFTWARE OPERATION QUALITY ASSURANCE



- Software operation contract review
- Software operation plan
- Software operation progress control
- Software operation metrics
- Software operation services quality costs